

Universidad Nacional de Río Cuarto
Facultad de Ciencias Exactas, Físico-Químicas y Naturales
Departamento de Computación
Licenciatura en Ciencias de la Computación

Bases de Datos II

Trabajo Práctico N.º 3

Procedimientos Almacenados y Triggers

Alumno: Tomás Rodeghiero
Año: 2026

Resolución completa sobre los motores *MySQL 8+*, *PostgreSQL 14+* y *Oracle XE*, apoyada en las bases creadas en los Prácticos 1 y 2.

Índice

1. Introducción	2
2. Ejercicio 1: Cantidad máxima de ítems por factura	2
3. Ejercicio 2: Cotización máxima por banco (PostgreSQL)	6
4. Ejercicio 3: Saldo de cuenta bancaria (Oracle)	10
5. Ejercicio 4: Detección de facturas inconsistentes (MySQL)	15
6. Ejercicio 5: Triggers y autonumerado	17
6.1. a) Mayúsculas antes de insertar un producto	18
6.2. b) Auditoría de cambios en <code>producto.cantidad</code>	19
6.3. c) Autonumerado de <code>parquimetro</code>	21
Conclusión	22

1. Introducción

El Práctico 3 introduce dos mecanismos centrales del lado servidor: **procedimientos y funciones almacenadas** y **triggers**. Ambos permiten desplazar lógica de negocio desde la aplicación hasta el motor, garantizando reglas de integridad y de auditoría que se cumplen *siempre*, sin depender del cliente que ejecute la operación. Los ejercicios reutilizan las bases del Práctico 1 (incisos 1.a y 1.b) y los usuarios definidos en el Práctico 2.

Cada ejercicio sigue siempre la misma estructura: **Consigna** (texto literal del PDF), **Idea** (lectura natural y concepto detrás), **Solución** (SQL en el motor que pide el enunciado), **Por qué funciona** (lógica paso a paso) y **En MySQL 8+** (equivalente portable a MySQL cuando el ejercicio pide otro motor). Cuando aplica, se agrega un bloque **Prueba** con sentencias de invocación.

Convenciones. Los nombres de bases siguen los del Práctico 1: `practico1a_mysql`, `practico1a` (esquema en PostgreSQL) y `practico1b_mysql`. En MySQL, los procedimientos y triggers se rodean con `DELIMITER $$...$$ DELIMITER ;` para que el cliente no corte la sentencia en cada `;` interno. Variables locales con prefijo `v_`; parámetros con prefijo `p_`.

2. Ejercicio 1: Cantidad máxima de ítems por factura

Consigna

Teniendo en cuenta el ejercicio 1 a) de la práctica Nro 1 (base de datos de facturación), agregue a la tabla `factura` el campo cantidad máxima de ítems permitidos. Controlar que la base de datos no acepte que una factura tenga más ítems que la cantidad de ítems permitidos por esta (Modifique la estructura de la tabla para que soporte la información de cuantos ítem puede tener cada factura).

Nota: Resolver utilizando los motores MySQL y PostgreSQL.

Idea

La consigna combina un cambio de estructura con una regla de negocio que *no* se puede expresar como un `CHECK` declarativo porque depende de otra tabla (la cantidad actual de filas en `item_factura`). Se resuelve con tres elementos: `ALTER TABLE` para agregar la columna, un trigger `BEFORE INSERT/UPDATE` sobre `item_factura` que cuenta ítems y aborta si supera el máximo, y un trigger `BEFORE UPDATE` sobre `factura` que impide bajar el límite por debajo de los ítems ya cargados.

Solución

MySQL 8+:

```
USE practico1a_mysql;

-- 1) Agregar la columna con default razonable y CHECK de rango.
ALTER TABLE factura
  ADD COLUMN cantidad_maxima_items INT NOT NULL DEFAULT 10,
  ADD CONSTRAINT ck_factura_cantidad_maxima_items
    CHECK (cantidad_maxima_items > 0);

-- 2) Trigger BEFORE INSERT en item_factura.
DROP TRIGGER IF EXISTS bi_item_factura_control_max_items;
DELIMITER $$
CREATE TRIGGER bi_item_factura_control_max_items
```

```

BEFORE INSERT ON item_factura
FOR EACH ROW
BEGIN
    DECLARE v_max INT;
    DECLARE v_actual INT;

    SELECT f.cantidad_maxima_items INTO v_max
    FROM factura f
    WHERE f.nro_factura = NEW.nro_factura;

    SELECT COUNT(*) INTO v_actual
    FROM item_factura i
    WHERE i.nro_factura = NEW.nro_factura;

    IF v_actual + 1 > v_max THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'La factura supera la cantidad maxima de items permitidos';
    END IF;
END$$
DELIMITER ;

-- 3) Trigger BEFORE UPDATE en item_factura: solo valida cambios de factura destino.
DROP TRIGGER IF EXISTS bu_item_factura_control_max_items;
DELIMITER $$
CREATE TRIGGER bu_item_factura_control_max_items
BEFORE UPDATE ON item_factura
FOR EACH ROW
BEGIN
    DECLARE v_max INT;
    DECLARE v_actual INT;

    IF NEW.nro_factura <> OLD.nro_factura THEN
        SELECT f.cantidad_maxima_items INTO v_max
        FROM factura f
        WHERE f.nro_factura = NEW.nro_factura;

        SELECT COUNT(*) INTO v_actual
        FROM item_factura i
        WHERE i.nro_factura = NEW.nro_factura;

        IF v_actual + 1 > v_max THEN
            SIGNAL SQLSTATE '45000'
            SET MESSAGE_TEXT = 'La factura destino supera la cantidad maxima de items
permitidos';
        END IF;
    END IF;
END IF;
END$$
DELIMITER ;

-- 4) Trigger BEFORE UPDATE en factura: impide bajar el maximo
-- por debajo de los items ya cargados.
DROP TRIGGER IF EXISTS bu_factura_control_max_items;
DELIMITER $$
CREATE TRIGGER bu_factura_control_max_items
BEFORE UPDATE ON factura
FOR EACH ROW
BEGIN
    DECLARE v_actual INT;

    IF NEW.cantidad_maxima_items <= 0 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'cantidad_maxima_items debe ser mayor que 0';
    END IF;

```

```

IF NEW.cantidad_maxima_items <> OLD.cantidad_maxima_items THEN
    SELECT COUNT(*) INTO v_actual
    FROM item_factura i
    WHERE i.nro_factura = OLD.nro_factura;

    IF v_actual > NEW.cantidad_maxima_items THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'No se puede reducir el maximo: ya hay mas items cargados';
    END IF;
END IF;
END$$
DELIMITER ;

```

PostgreSQL 14+: este motor separa la *función* (lógica) del *trigger* (disparador). Una sola función cubre INSERT y UPDATE con TG_OP.

```

SET search_path TO practico1a;

ALTER TABLE practico1a.factura
    ADD COLUMN cantidad_maxima_items INTEGER NOT NULL DEFAULT 10;

ALTER TABLE practico1a.factura
    ADD CONSTRAINT ck_factura_cantidad_maxima_items
    CHECK (cantidad_maxima_items > 0);

-- Funcion + trigger sobre item_factura
DROP TRIGGER IF EXISTS biu_item_factura_control_max_items
    ON practico1a.item_factura;
DROP FUNCTION IF EXISTS practico1a.fn_control_max_items_item_factura();

CREATE OR REPLACE FUNCTION practico1a.fn_control_max_items_item_factura()
RETURNS TRIGGER
LANGUAGE plpgsql
AS $$
DECLARE
    v_max INTEGER;
    v_actual INTEGER;
BEGIN
    IF TG_OP = 'UPDATE' AND NEW.nro_factura = OLD.nro_factura THEN
        RETURN NEW;
    END IF;

    SELECT f.cantidad_maxima_items INTO v_max
    FROM practico1a.factura f
    WHERE f.nro_factura = NEW.nro_factura
    FOR UPDATE;

    SELECT COUNT(*) INTO v_actual
    FROM practico1a.item_factura i
    WHERE i.nro_factura = NEW.nro_factura;

    IF v_actual + 1 > v_max THEN
        RAISE EXCEPTION
            'La factura % supera la cantidad maxima de items permitidos (%).',
            NEW.nro_factura, v_max;
    END IF;

    RETURN NEW;
END;
$$;

```

```

CREATE TRIGGER biu_item_factura_control_max_items
BEFORE INSERT OR UPDATE ON practico1a.item_factura
FOR EACH ROW
EXECUTE FUNCTION practico1a.fn_control_max_items_item_factura();

-- Funcion + trigger sobre factura.cantidad_maxima_items
DROP TRIGGER IF EXISTS bu_factura_control_max_items
ON practico1a.factura;
DROP FUNCTION IF EXISTS practico1a.fn_control_update_max_items_factura();

CREATE OR REPLACE FUNCTION practico1a.fn_control_update_max_items_factura()
RETURNS TRIGGER
LANGUAGE plpgsql
AS $$
DECLARE
    v_actual INTEGER;
BEGIN
    IF NEW.cantidad_maxima_items <= 0 THEN
        RAISE EXCEPTION 'cantidad_maxima_items debe ser mayor que 0';
    END IF;

    IF NEW.cantidad_maxima_items <> OLD.cantidad_maxima_items THEN
        SELECT COUNT(*) INTO v_actual
        FROM practico1a.item_factura i
        WHERE i.nro_factura = OLD.nro_factura;

        IF v_actual > NEW.cantidad_maxima_items THEN
            RAISE EXCEPTION
                'No se puede reducir el maximo de la factura %: tiene % items cargados',
                OLD.nro_factura, v_actual;
        END IF;
    END IF;

    RETURN NEW;
END;
$$;

CREATE TRIGGER bu_factura_control_max_items
BEFORE UPDATE OF cantidad_maxima_items ON practico1a.factura
FOR EACH ROW
EXECUTE FUNCTION practico1a.fn_control_update_max_items_factura();

```

Por qué funciona

- **Tres triggers:** el de INSERT cubre las altas; el de UPDATE sobre `item_factura` solo actúa cuando el ítem se reasigna a otra factura (evita trabajo en updates inocuos de cantidad/-precio); y el de UPDATE sobre `factura` cubre el caso simétrico, en el que el problema no es el ítem que entra sino el límite que baja.
- Para *abortar* la operación: en MySQL se usa `SIGNAL SQLSTATE '45000'` con un `MESSAGE_TEXT`; en PostgreSQL se usa `RAISE EXCEPTION`. Ambos cancelan la transacción del cliente.
- En PostgreSQL, el `SELECT ... FOR UPDATE` sobre la fila de `factura` bloquea la fila para evitar condiciones de carrera: si dos sesiones intentan insertar el último ítem al mismo tiempo, una espera a que la otra confirme.
- `BEFORE UPDATE OF cantidad_maxima_items` (PostgreSQL) dispara solo cuando esa columna cambia, lo que es más eficiente que disparar en cualquier `UPDATE`.

En MySQL 8+

La consigna ya pide MySQL: la sección *Solución* contiene la versión MySQL 8+. Las únicas

diferencias relevantes con PostgreSQL son sintácticas: `DECLARE` de variables dentro del cuerpo del trigger, `SIGNAL SQLSTATE '45000'` en vez de `RAISE EXCEPTION`, y la imposibilidad de combinar varios eventos en un solo trigger (en MySQL hay un trigger por evento; en PostgreSQL un solo `BEFORE INSERT OR UPDATE` cubre dos eventos).

Prueba

Las pruebas son válidas para ambos motores (cambian solo los nombres de esquema).

```
-- Suponiendo que la factura 1001 ya tiene 2 items cargados.
UPDATE factura SET cantidad_maxima_items = 2 WHERE nro_factura = 1001;

-- FALLA: dejaria 3 items en la factura 1001.
INSERT INTO item_factura (cod_producto, nro_factura, cantidad, precio)
VALUES (101, 1001, 1, 15000.00);

-- FALLA: intenta reducir el maximo por debajo de los 2 items existentes.
UPDATE factura SET cantidad_maxima_items = 1 WHERE nro_factura = 1001;

-- FUNCIONA: subir el maximo siempre es valido.
UPDATE factura SET cantidad_maxima_items = 5 WHERE nro_factura = 1001;
```

3. Ejercicio 2: Cotización máxima por banco (PostgreSQL)

Consigna

Utilizando el motor de base de datos PostgreSQL, cree las tablas:

```
Banco (cod_banco, nombre, precio, cotizacion_maxima)
Cotización (cod_cotizacion, cod_banco, cotización)
Cod_banco: clave foránea a Banco
```

Realizar una función que obtenga la máxima cotización para un banco dado y actualice el valor en la tabla Banco, la función debe retornar la cotización máxima en un parámetro de salida.

Nota: No utilizar la función `MAX`!!!

Idea

Es una función con *parámetros mixtos* IN y OUT: recibe el código de banco, calcula la cotización máxima recorriendo manualmente las filas (sin usar `MAX`) y persiste el resultado en `Banco.cotizacion_maxima`. El concepto clave es el patrón de cálculo iterativo con `FOR ... LOOP` en PL/pgSQL: una variable acumuladora se actualiza cuando aparece un valor mayor al visto hasta el momento.

Solución

Esquema y tablas:

```
CREATE SCHEMA IF NOT EXISTS practico3_ej2;
SET search_path TO practico3_ej2;

DROP TABLE IF EXISTS cotizacion;
DROP TABLE IF EXISTS banco;

CREATE TABLE banco (
    cod_banco INTEGER PRIMARY KEY,
    nombre VARCHAR(80) NOT NULL,
    precio NUMERIC(12,4) NOT NULL,
```

```

cotizacion_maxima NUMERIC(12,4),
CONSTRAINT ck_banco_precio CHECK (precio > 0),
CONSTRAINT ck_banco_cot_max
    CHECK (cotizacion_maxima IS NULL OR cotizacion_maxima > 0)
);

CREATE TABLE cotizacion (
    cod_cotizacion INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    cod_banco INTEGER NOT NULL,
    cotizacion NUMERIC(12,4) NOT NULL,
    CONSTRAINT ck_cotizacion_valor CHECK (cotizacion > 0),
    CONSTRAINT fk_cotizacion_banco
        FOREIGN KEY (cod_banco) REFERENCES banco(cod_banco)
        ON DELETE RESTRICT
);

CREATE INDEX idx_cotizacion_cod_banco ON cotizacion(cod_banco);

```

Función con parámetros IN/OUT:

```

CREATE OR REPLACE FUNCTION practico3_ej2.fn_actualizar_cotizacion_maxima(
    IN p_cod_banco INTEGER,
    OUT p_cotizacion_maxima NUMERIC(12,4)
)
LANGUAGE plpgsql
AS $$
DECLARE
    v_cot NUMERIC(12,4);
BEGIN
    -- Verificación de existencia + bloqueo del banco.
    PERFORM 1
    FROM practico3_ej2.banco b
    WHERE b.cod_banco = p_cod_banco
    FOR UPDATE;

    IF NOT FOUND THEN
        RAISE EXCEPTION 'No existe banco con cod_banco=%', p_cod_banco;
    END IF;

    p_cotizacion_maxima := NULL;

    -- Calculo manual del maximo, sin MAX.
    FOR v_cot IN
        SELECT c.cotizacion
        FROM practico3_ej2.cotizacion c
        WHERE c.cod_banco = p_cod_banco
    LOOP
        IF p_cotizacion_maxima IS NULL OR v_cot > p_cotizacion_maxima THEN
            p_cotizacion_maxima := v_cot;
        END IF;
    END LOOP;

    IF p_cotizacion_maxima IS NULL THEN
        RAISE EXCEPTION 'El banco % no tiene cotizaciones cargadas', p_cod_banco;
    END IF;

    -- Persistir el resultado en la tabla banco.
    UPDATE practico3_ej2.banco b
    SET cotizacion_maxima = p_cotizacion_maxima
    WHERE b.cod_banco = p_cod_banco;

    RETURN;
END;

```



```
$$;
```

Por qué funciona

- **Parámetros:** p_cod_banco entra como IN, p_cotizacion_maxima sale como OUT. PostgreSQL trata las funciones con OUT como devolviendo una fila con esas columnas; por eso se invoca con `SELECT * FROM fn(...)`.
- **Iteración manual:** el `FOR v_cot IN SELECT ... LOOP` avanza una fila por iteración. La variable p_cotizacion_maxima se inicializa en NULL y se actualiza con cada valor mayor. Así se respeta la restricción de no usar MAX.
- **Manejo de errores:** dos `RAISE EXCEPTION` distintos (banco inexistente vs. banco sin cotizaciones) con mensajes diferentes para que el cliente pueda diferenciarlos.
- **Concurrencia:** `PERFORM ... FOR UPDATE` bloquea la fila del banco; dos invocaciones simultáneas no se pisan al actualizar cotizacion_maxima.

En MySQL 8+

MySQL 8+ no permite parámetros OUT en FUNCTION (solo en PROCEDURE); la traducción idiomática es un PROCEDURE con IN p_cod_banco y OUT p_cotizacion_maxima. El recorrido iterativo se hace con un CURSOR y un CONTINUE HANDLER FOR NOT FOUND, que es el patrón estándar en MySQL para iterar sin agregaciones.

```
USE practico3_ej2_mysql;  -- equivalente del schema practico3_ej2

DROP TABLE IF EXISTS cotizacion;
DROP TABLE IF EXISTS banco;

CREATE TABLE banco (
    cod_banco INT PRIMARY KEY,
    nombre VARCHAR(80) NOT NULL,
    precio DECIMAL(12,4) NOT NULL,
    cotizacion_maxima DECIMAL(12,4) NULL,
    CONSTRAINT ck_banco_precio CHECK (precio > 0)
);

CREATE TABLE cotizacion (
    cod_cotizacion INT AUTO_INCREMENT PRIMARY KEY,
    cod_banco INT NOT NULL,
    cotizacion DECIMAL(12,4) NOT NULL,
    CONSTRAINT ck_cotizacion_valor CHECK (cotizacion > 0),
    CONSTRAINT fk_cotizacion_banco
        FOREIGN KEY (cod_banco) REFERENCES banco(cod_banco)
);

DROP PROCEDURE IF EXISTS sp_actualizar_cotizacion_maxima;
DELIMITER $$
CREATE PROCEDURE sp_actualizar_cotizacion_maxima(
    IN p_cod_banco INT,
    OUT p_cotizacion_maxima DECIMAL(12,4)
)
BEGIN
    DECLARE v_cot DECIMAL(12,4);
    DECLARE v_done INT DEFAULT FALSE;
    DECLARE v_existe INT;

    DECLARE cur_cot CURSOR FOR
        SELECT cotizacion
        FROM cotizacion
        WHERE cod_banco = p_cod_banco;
```

```

DECLARE CONTINUE HANDLER FOR NOT FOUND SET v_done = TRUE;

SELECT COUNT(*) INTO v_existe FROM banco WHERE cod_banco = p_cod_banco;
IF v_existe = 0 THEN
    SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'No existe banco con ese codigo';
END IF;

SET p_cotizacion_maxima = NULL;

OPEN cur_cot;
read_loop: LOOP
    FETCH cur_cot INTO v_cot;
    IF v_done THEN LEAVE read_loop; END IF;
    IF p_cotizacion_maxima IS NULL OR v_cot > p_cotizacion_maxima THEN
        SET p_cotizacion_maxima = v_cot;
    END IF;
END LOOP;
CLOSE cur_cot;

IF p_cotizacion_maxima IS NULL THEN
    SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'El banco no tiene cotizaciones cargadas';
END IF;

UPDATE banco
    SET cotizacion_maxima = p_cotizacion_maxima
    WHERE cod_banco = p_cod_banco;
END$$
DELIMITER ;

-- Invocacion
CALL sp_actualizar_cotizacion_maxima(1, @max);
SELECT @max AS cotizacion_maxima;

```

Qué cambia respecto de PostgreSQL: (i) FUNCTION sin OUT en MySQL, se convierte a PROCEDURE; (ii) el FOR ... LOOP sobre un SELECT no existe directamente: se reemplaza por CURSOR + CONTINUE HANDLER FOR NOT FOUND; (iii) RAISE EXCEPTION se vuelve SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT.

Prueba

```

-- PostgreSQL
SET search_path TO practico3_ej2;

INSERT INTO banco (cod_banco, nombre, precio, cotizacion_maxima) VALUES
    (1, 'Banco Rio', 1200.0000, NULL),
    (2, 'Banco Sur', 950.0000, NULL);

INSERT INTO cotizacion (cod_banco, cotizacion) VALUES
    (1, 1210.5000), (1, 1198.2000), (1, 1245.7500), (1, 1222.1000),
    (2, 960.0000), (2, 1001.3000), (2, 998.9000);

-- Maximos esperados: 1245.7500 para el banco 1, 1001.3000 para el 2.
SELECT * FROM practico3_ej2.fn_actualizar_cotizacion_maxima(1);
SELECT * FROM practico3_ej2.fn_actualizar_cotizacion_maxima(2);

SELECT cod_banco, nombre, cotizacion_maxima
FROM practico3_ej2.banco
ORDER BY cod_banco;

```

4. Ejercicio 3: Saldo de cuenta bancaria (Oracle)

Consigna

Utilizando el motor de base de datos Oracle, cree las tablas:

```
cuenta (nro_cuenta, saldo)
movimiento (nro_movimiento, nro_cuenta, fecha, debe, haber)
nro_cuenta: Clave foránea a cuenta
```

Luego, cree los siguientes procedimientos almacenados:

- Que inserte un movimiento y mantenga consistente el saldo en la tabla cuenta. El procedimiento debe tomar como parámetro el nro de cuenta, el debe y el haber. Al final del procedimiento imprima por pantalla el número de cuenta y el saldo actual.
- Que calcule el saldo de una cuenta a una determinada fecha. El procedimiento, además de mostrar por pantalla el saldo, lo debe retornar en un parámetro.

Idea

Dos procedimientos PL/SQL con patrones clásicos: el primero es un *procedimiento transaccional* con validaciones, `SELECT ... FOR UPDATE` para concurrencia, `INSERT + UPDATE` en la misma transacción y salida con `DBMS_OUTPUT.PUT_LINE`. El segundo es un *cálculo retrospectivo*: parte del saldo actual y le resta el efecto neto de los movimientos posteriores a la fecha dada, retornándolo por un parámetro OUT. El saldo se mantiene denormalizado en `cuenta.saldo` para no recalcularlo desde el principio cada vez.

Solución

Esquema:

```
SET SERVEROUTPUT ON;

CREATE TABLE cuenta (
  nro_cuenta NUMBER(10) CONSTRAINT pk_cuenta PRIMARY KEY,
  saldo NUMBER(14,2) DEFAULT 0 NOT NULL
);

CREATE TABLE movimiento (
  nro_movimiento NUMBER(12) CONSTRAINT pk_movimiento PRIMARY KEY,
  nro_cuenta NUMBER(10) NOT NULL,
  fecha DATE DEFAULT SYSDATE NOT NULL,
  debe NUMBER(14,2) DEFAULT 0 NOT NULL,
  haber NUMBER(14,2) DEFAULT 0 NOT NULL,
  CONSTRAINT fk_movimiento_cuenta
    FOREIGN KEY (nro_cuenta) REFERENCES cuenta(nro_cuenta),
  CONSTRAINT ck_mov_montos_no_negativos
    CHECK (debe >= 0 AND haber >= 0),
  CONSTRAINT ck_mov_operacion_valida
    CHECK ((debe > 0 AND haber = 0) OR (haber > 0 AND debe = 0))
);

-- Oracle 10g/11g no tiene IDENTITY: secuencia + trigger.
CREATE SEQUENCE seq_movimiento
  START WITH 1 INCREMENT BY 1 NOCACHE NOCYCLE;

CREATE OR REPLACE TRIGGER bi_movimiento
BEFORE INSERT ON movimiento
FOR EACH ROW
BEGIN
```

```

IF :NEW.nro_movimiento IS NULL THEN
    SELECT seq_movimiento.NEXTVAL INTO :NEW.nro_movimiento FROM dual;
END IF;

IF :NEW.fecha IS NULL THEN
    :NEW.fecha := SYSDATE;
END IF;
END;
/

```

Inciso a) Procedimiento que inserta movimiento y mantiene saldo:

```

CREATE OR REPLACE PROCEDURE sp_registrar_movimiento (
    p_nro_cuenta IN cuenta.nro_cuenta%TYPE,
    p_debe       IN NUMBER,
    p_haber      IN NUMBER
)
IS
    v_saldo_actual cuenta.saldo%TYPE;
    v_debe  NUMBER(14,2) := NVL(p_debe, 0);
    v_haber NUMBER(14,2) := NVL(p_haber, 0);
BEGIN
    IF v_debe < 0 OR v_haber < 0 THEN
        RAISE_APPLICATION_ERROR(-20001,
            'Debe/Haber no pueden ser negativos');
    END IF;

    IF NOT ((v_debe > 0 AND v_haber = 0)
        OR (v_haber > 0 AND v_debe = 0)) THEN
        RAISE_APPLICATION_ERROR(-20002,
            'La operacion debe tener solo DEBE o solo HABER');
    END IF;

    -- Bloqueo de la cuenta para consistencia bajo concurrencia.
    SELECT c.saldo INTO v_saldo_actual
    FROM cuenta c
    WHERE c.nro_cuenta = p_nro_cuenta
    FOR UPDATE;

    INSERT INTO movimiento (nro_cuenta, fecha, debe, haber)
    VALUES (p_nro_cuenta, SYSDATE, v_debe, v_haber);

    UPDATE cuenta c
    SET c.saldo = c.saldo + v_debe - v_haber
    WHERE c.nro_cuenta = p_nro_cuenta
    RETURNING c.saldo INTO v_saldo_actual;

    DBMS_OUTPUT.PUT_LINE(
        'Cuenta: ' || p_nro_cuenta || ' | Saldo actual: ' ||
        TO_CHAR(v_saldo_actual, 'FM9999999990D00')
    );
EXCEPTION
    WHEN NO_DATA_FOUND THEN
        RAISE_APPLICATION_ERROR(-20003,
            'No existe la cuenta ' || p_nro_cuenta);
END;
/

```

Inciso b) Saldo a una fecha dada:

```

CREATE OR REPLACE PROCEDURE sp_saldo_a_fecha (
    p_nro_cuenta IN cuenta.nro_cuenta%TYPE,

```

```

    p_fecha      IN DATE,
    p_saldo      OUT NUMBER
)
IS
    v_saldo_actual  cuenta.saldo %TYPE;
    v_delta_posterior NUMBER(14,2);
BEGIN
    SELECT c.saldo INTO v_saldo_actual
    FROM cuenta c
    WHERE c.nro_cuenta = p_nro_cuenta;

    -- Impacto neto de los movimientos POSTERIORES a p_fecha.
    SELECT NVL(SUM(m.debe - m.haber), 0)
    INTO v_delta_posterior
    FROM movimiento m
    WHERE m.nro_cuenta = p_nro_cuenta
    AND m.fecha > p_fecha;

    p_saldo := v_saldo_actual - v_delta_posterior;

    DBMS_OUTPUT.PUT_LINE(
        'Cuenta: ' || p_nro_cuenta ||
        ' | Fecha: ' || TO_CHAR(p_fecha, 'YYYY-MM-DD HH24:MI:SS') ||
        ' | Saldo: ' || TO_CHAR(p_saldo, 'FM999999990D00')
    );
EXCEPTION
    WHEN NO_DATA_FOUND THEN
        RAISE_APPLICATION_ERROR(-20004,
            'No existe la cuenta ' || p_nro_cuenta);
END;
/

```

Por qué funciona

- En el inciso a), `NVL(p_debe, 0)` normaliza nulos a cero; las dos validaciones de negocio se traducen en `RAISE_APPLICATION_ERROR` con códigos distintos (-20001, -20002) para que el cliente pueda diferenciarlas.
- `SELECT ... FOR UPDATE` sobre la cuenta evita condiciones de carrera bajo concurrencia: dos movimientos sobre la misma cuenta se serializan.
- `RETURNING c.saldo INTO v_saldo_actual` captura el saldo nuevo justo después del `UPDATE`, ahorrando una consulta extra. `DBMS_OUTPUT.PUT_LINE` es el mecanismo de Oracle para escribir a la consola de sqlplus (requiere `SET SERVEROUTPUT ON`).
- En el inciso b), la idea es: `saldo_a_fecha = saldo_actual - (suma de debe-haber de los movimientos posteriores)`. Es equivalente a sumar desde el principio hasta `p_fecha`, pero más barato cuando hay mucha historia antigua.
- El bloque `EXCEPTION WHEN NO_DATA_FOUND` atrapa el caso de cuenta inexistente: `SELECT INTO` lanza esa excepción automáticamente si no encuentra filas.

En MySQL 8+

MySQL 8+ traduce ambos procedimientos directo, con tres diferencias: (i) no existe `%TYPE`, hay que escribir el tipo concreto; (ii) no existe `DBMS_OUTPUT`, la salida se hace con `SELECT`; (iii) no existe `RAISE_APPLICATION_ERROR`, se usa `SIGNAL SQLSTATE '45000'` con `MESSAGE_TEXT`. `RETURNING` tampoco existe en MySQL: hay que hacer un `SELECT` extra para leer el saldo después del `UPDATE`.

```
USE practico3_ej3_mysql;
```

```

CREATE TABLE cuenta (
    nro_cuenta INT PRIMARY KEY,
    saldo DECIMAL(14,2) NOT NULL DEFAULT 0
);

CREATE TABLE movimiento (
    nro_movimiento INT AUTO_INCREMENT PRIMARY KEY,
    nro_cuenta INT NOT NULL,
    fecha DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
    debe DECIMAL(14,2) NOT NULL DEFAULT 0,
    haber DECIMAL(14,2) NOT NULL DEFAULT 0,
    CONSTRAINT fk_movimiento_cuenta
        FOREIGN KEY (nro_cuenta) REFERENCES cuenta(nro_cuenta),
    CONSTRAINT ck_mov_montos CHECK (debe >= 0 AND haber >= 0),
    CONSTRAINT ck_mov_operacion
        CHECK ((debe > 0 AND haber = 0) OR (haber > 0 AND debe = 0))
);

-- Inciso a)
DROP PROCEDURE IF EXISTS sp_registrar_movimiento;
DELIMITER $$
CREATE PROCEDURE sp_registrar_movimiento(
    IN p_nro_cuenta INT,
    IN p_debe DECIMAL(14,2),
    IN p_haber DECIMAL(14,2)
)
BEGIN
    DECLARE v_existe INT;
    DECLARE v_saldo DECIMAL(14,2);
    DECLARE v_debe DECIMAL(14,2) DEFAULT IFNULL(p_debe, 0);
    DECLARE v_haber DECIMAL(14,2) DEFAULT IFNULL(p_haber, 0);

    IF v_debe < 0 OR v_haber < 0 THEN
        SIGNAL SQLSTATE '45000'
            SET MESSAGE_TEXT = 'Debe/Haber no pueden ser negativos';
    END IF;

    IF NOT ((v_debe > 0 AND v_haber = 0)
        OR (v_haber > 0 AND v_debe = 0)) THEN
        SIGNAL SQLSTATE '45000'
            SET MESSAGE_TEXT = 'La operacion debe tener solo DEBE o solo HABER';
    END IF;

    SELECT COUNT(*) INTO v_existe FROM cuenta WHERE nro_cuenta = p_nro_cuenta;
    IF v_existe = 0 THEN
        SIGNAL SQLSTATE '45000'
            SET MESSAGE_TEXT = 'No existe la cuenta';
    END IF;

    -- Bloqueo de la cuenta (requiere transaccion explicita en InnoDB).
    SELECT saldo INTO v_saldo
    FROM cuenta
    WHERE nro_cuenta = p_nro_cuenta
    FOR UPDATE;

    INSERT INTO movimiento (nro_cuenta, debe, haber)
    VALUES (p_nro_cuenta, v_debe, v_haber);

    UPDATE cuenta SET saldo = saldo + v_debe - v_haber
    WHERE nro_cuenta = p_nro_cuenta;

    -- Reemplazo de DBMS_OUTPUT: SELECT como salida visible para el cliente.

```

```

SELECT nro_cuenta, saldo
  FROM cuenta
 WHERE nro_cuenta = p_nro_cuenta;
END$$
DELIMITER ;

-- Inciso b)
DROP PROCEDURE IF EXISTS sp_saldo_a_fecha;
DELIMITER $$
CREATE PROCEDURE sp_saldo_a_fecha(
  IN  p_nro_cuenta INT,
  IN  p_fecha      DATETIME,
  OUT p_saldo      DECIMAL(14,2)
)
BEGIN
  DECLARE v_saldo_actual    DECIMAL(14,2);
  DECLARE v_delta_posterior DECIMAL(14,2);
  DECLARE v_existe          INT;

  SELECT COUNT(*) INTO v_existe FROM cuenta WHERE nro_cuenta = p_nro_cuenta;
  IF v_existe = 0 THEN
    SIGNAL SQLSTATE '45000'
      SET MESSAGE_TEXT = 'No existe la cuenta';
  END IF;

  SELECT saldo INTO v_saldo_actual FROM cuenta WHERE nro_cuenta = p_nro_cuenta;

  SELECT IFNULL(SUM(debe - haber), 0) INTO v_delta_posterior
    FROM movimiento
   WHERE nro_cuenta = p_nro_cuenta
     AND fecha > p_fecha;

  SET p_saldo = v_saldo_actual - v_delta_posterior;

  -- "Mostrar por pantalla" via SELECT.
  SELECT p_nro_cuenta AS nro_cuenta, p_fecha AS fecha, p_saldo AS saldo;
END$$
DELIMITER ;

-- Invocacion
CALL sp_registrar_movimiento(1001, 1000, 0);
CALL sp_saldo_a_fecha(1001, NOW(), @s);
SELECT @s AS saldo_retornado;

```

Qué cambia respecto de Oracle: (i) sin %TYPE hay que repetir el tipo; (ii) la "salida por pantalla" se simula con SELECT, porque MySQL no tiene buffer de salida tipo DBMS_OUTPUT; (iii) RETURNING ... INTO se reemplaza por un SELECT extra; (iv) errores con SIGNAL SQLSTATE '45000' en lugar de RAISE_APPLICATION_ERROR.

Prueba

```

-- Oracle
SET SERVEROUTPUT ON;

INSERT INTO cuenta (nro_cuenta, saldo) VALUES (1001, 0);
INSERT INTO cuenta (nro_cuenta, saldo) VALUES (1002, 500);

BEGIN
  sp_registrar_movimiento(1001, 1000, 0); -- saldo 1000
  sp_registrar_movimiento(1001, 0, 150);  -- saldo 850
  sp_registrar_movimiento(1001, 200, 0);  -- saldo 1050
END;

```

```

/

DECLARE
    v_saldo NUMBER;
BEGIN
    sp_saldo_a_fecha(1001, SYSDATE, v_saldo);
    DBMS_OUTPUT.PUT_LINE('Saldo retornado por OUT: ' ||
        TO_CHAR(v_saldo, 'FM999999990D00'));
END;
/

```

5. Ejercicio 4: Detección de facturas inconsistentes (MySQL)

Consigna

Utilizando el motor MySQL con la base de datos definida en el ejercicio 1a) de la Práctica 1 de Repaso de SQL (Base de datos de clientes y productos), y haciéndole las modificaciones que crea necesarias, cree un procedimiento que guarde información en una tabla con las facturas que no son consistentes (una factura no es consistente si la suma de sus ítems es diferente al monto total de la factura). Considerar:

- El procedimiento debe ser creado con el usuario vendedor2 (creado en la práctica anterior) y el procedimiento debe ser creado para ejecutarse con los permisos del invocador.
- Ejecute el procedimiento con los usuarios vendedor2 y vendedor1.

Idea

Dos conceptos del lado servidor entran en juego: (1) un procedimiento que materializa en una tabla `facturas_inconsistentes` todas las facturas cuyo monto difiere de la suma de `cantidad * precio` de sus ítems; (2) `SQL SECURITY INVOKER`, que hace que el procedimiento se ejecute con los permisos del invocador (no del creador). Eso obliga a configurar los `GRANT` necesarios para cada usuario que vaya a llamarlo, conectando el Práctico 3 con el DCL del Práctico 2.

Solución

Preparación (como root): En el Práctico 2 (inciso 1.f) se le revocaron todos los privilegios a `vendedor2`; hay que reotorgarle los mínimos para crear el procedimiento, y como correrá con `INVOKER`, también hay que dotar a `vendedor1` de los privilegios que la rutina ejecuta.

```

USE practico1a_mysql;

-- Tabla destino donde se guardan las inconsistencias detectadas.
CREATE TABLE IF NOT EXISTS facturas_inconsistentes (
    nro_factura        INT PRIMARY KEY,
    monto_factura      DECIMAL(12,2) NOT NULL,
    monto_items        DECIMAL(12,2) NOT NULL,
    diferencia         DECIMAL(12,2) NOT NULL,
    fecha_control      DATETIME NOT NULL,
    usuario_ejecucion  VARCHAR(100) NOT NULL
);

-- Permiso minimo para que vendedor2 pueda crear procedimientos.
GRANT CREATE ROUTINE ON practico1a_mysql.*
    TO 'vendedor2'@'localhost';

-- El procedimiento corre con SQL SECURITY INVOKER, asi que cada usuario
-- que lo ejecute necesita estos permisos:

```



```
GRANT SELECT (nro_factura, monto)
ON practico1a_mysql.factura
TO 'vendedor1'@'localhost', 'vendedor2'@'localhost';

GRANT SELECT (nro_factura, cantidad, precio)
ON practico1a_mysql.item_factura
TO 'vendedor1'@'localhost', 'vendedor2'@'localhost';

GRANT INSERT, UPDATE, SELECT
ON practico1a_mysql.facturas_inconsistentes
TO 'vendedor1'@'localhost', 'vendedor2'@'localhost';
```

Creación del procedimiento (conectado como *vendedor2*):

```
-- $ mysql -u vendedor2 -p practico1a_mysql

DROP PROCEDURE IF EXISTS sp_guardar_facturas_inconsistentes;
DELIMITER $$
CREATE PROCEDURE sp_guardar_facturas_inconsistentes()
SQL SECURITY INVOKER
BEGIN
    INSERT INTO facturas_inconsistentes (
        nro_factura, monto_factura, monto_items,
        diferencia, fecha_control, usuario_ejecucion
    )
    SELECT
        f.nro_factura,
        f.monto AS monto_factura,
        ROUND(COALESCE(SUM(i.cantidad * i.precio), 0), 2) AS monto_items,
        ROUND(COALESCE(SUM(i.cantidad * i.precio), 0) - f.monto, 2)
            AS diferencia,
        NOW() AS fecha_control,
        USER() AS usuario_ejecucion
    FROM factura f
    LEFT JOIN item_factura i ON i.nro_factura = f.nro_factura
    GROUP BY f.nro_factura, f.monto
    HAVING ROUND(COALESCE(SUM(i.cantidad * i.precio), 0), 2)
        <> ROUND(f.monto, 2)
    ON DUPLICATE KEY UPDATE
        monto_factura = VALUES(monto_factura),
        monto_items = VALUES(monto_items),
        diferencia = VALUES(diferencia),
        fecha_control = VALUES(fecha_control),
        usuario_ejecucion = VALUES(usuario_ejecucion);
END$$
DELIMITER ;
```

Permiso de ejecución (como *root*):

```
GRANT EXECUTE ON PROCEDURE practico1a_mysql.sp_guardar_facturas_inconsistentes
TO 'vendedor1'@'localhost', 'vendedor2'@'localhost';
```

Verificación del DEFINER:

```
SHOW CREATE PROCEDURE sp_guardar_facturas_inconsistentes\G
-- En la salida debe verse:
-- DEFINER = 'vendedor2'@'localhost'
-- SQL SECURITY INVOKER
```

Por qué funciona

- Por defecto MySQL crea procedimientos con `SQL SECURITY DEFINER`: corren con los permisos de quien los creó. La consigna pide lo contrario (`INVOKER`), que obliga a que cada llamador tenga sus propios permisos. De ahí los `GRANT` sobre `factura`, `item_factura` y la tabla destino para ambos vendedores.
- El `LEFT JOIN` entre `factura` e `item_factura` permite que aparezcan también facturas sin ítems (suma 0). `COALESCE` convierte los `NULL` de la suma vacía en 0 para que la comparación contra `monto` tenga sentido.
- `HAVING ROUND(...) <> ROUND(f.monto, 2)` filtra después de la agregación. El `ROUND` a dos decimales evita falsos positivos por residuos de coma flotante.
- `ON DUPLICATE KEY UPDATE` reemplaza el registro si la factura ya estaba marcada como inconsistente: re-ejecutar el procedimiento no falla por clave duplicada, simplemente refresca la lectura. `USER()` devuelve el usuario realmente conectado, así se distingue quién corrió el control.

En MySQL 8+

La consigna ya pide MySQL: la *Solución* es la versión MySQL 8+. Vale anotar la portabilidad inversa: en PostgreSQL, el equivalente de `SQL SECURITY INVOKER/DEFINER` se llama `SECURITY INVOKER/SECURITY DEFINER` dentro del `CREATE FUNCTION/PROCEDURE`; en Oracle es `AUTHID CURRENT_USER (invoker)` vs `AUTHID DEFINER (default)`. El uso de `ON DUPLICATE KEY UPDATE` es propio de MySQL; en PostgreSQL se hace con `INSERT ... ON CONFLICT ... DO UPDATE`, y en Oracle con `MERGE INTO`.

Prueba

```
-- Forzar una inconsistencia (solo root suele tener UPDATE sobre factura):
USE practico1a_mysql;
UPDATE factura SET monto = monto + 100 WHERE nro_factura = 1001;

-- Conectado como vendedor2:
-- $ mysql -u vendedor2 -p practico1a_mysql
CALL sp_guardar_facturas_inconsistentes();
SELECT * FROM facturas_inconsistentes
ORDER BY fecha_control DESC, nro_factura;

-- Conectado como vendedor1:
-- $ mysql -u vendedor1 -p practico1a_mysql
CALL sp_guardar_facturas_inconsistentes();
SELECT * FROM facturas_inconsistentes
ORDER BY fecha_control DESC, nro_factura;
```

6. Ejercicio 5: Triggers y autonumerado

Consigna

Cree triggers teniendo en cuenta las siguientes consideraciones:

- En la base de datos creada en el ejercicio 1a) de la Práctica 1) de Repaso de SQL, agregue un trigger para que antes de cada inserción de un producto se guarde la descripción en mayúsculas.
- En la base de datos creada en el ejercicio 1a) de la Práctica 1) de Repaso de SQL, agregue un trigger, para que luego de cada modificación en la cantidad de un producto se guarde información de auditoria. Esta información debe quedar almacenada en la tabla auditoria-Producto (crearla antes de definir el trigger), la información a almacenar es `cod_producto`,

movimiento (diferencia de cantidades), fecha de realización de la actualización y el usuario que lo hizo. Con respecto al usuario que la realizó, debe ser el código de usuario del usuario logueado en la aplicación cliente de la base de datos. **Hint:** ver ejemplo Teórico.

- c) Modifique la base de datos creada en el inciso b) del ejercicio 1 de la Práctica 1 (bases de datos de estacionamiento) considerando que #parquimetro es auto numerado. ¿Cómo lo resolvería en Oracle? **Hint:** ver SEQUENCE.

Nota: Resolver utilizando los motores que se utilizaron en la creación de las bases de datos.

Idea

Tres usos distintos de trigger: (a) normalización antes de escribir (BEFORE INSERT); (b) auditoría después del cambio (AFTER UPDATE) usando una variable de sesión que provee el usuario de aplicación, ya que el motor no la conoce por sí mismo; (c) autonumerado, que en MySQL es directo con AUTO_INCREMENT, pero en Oracle 10/11 XE requiere el patrón clásico de SEQUENCE + TRIGGER BEFORE INSERT (no hay IDENTITY hasta Oracle 12c).

6.1. a) Mayúsculas antes de insertar un producto

Solución

MySQL 8+ (practico1a_mysql):

```
USE practico1a_mysql;

DROP TRIGGER IF EXISTS bi_producto_mayusculas;
DELIMITER $$
CREATE TRIGGER bi_producto_mayusculas
BEFORE INSERT ON producto
FOR EACH ROW
BEGIN
    SET NEW.descripcion = UPPER(TRIM(NEW.descripcion));
END$$
DELIMITER ;
```

PostgreSQL 14+ (practico1a):

```
SET search_path TO practico1a;

DROP TRIGGER IF EXISTS bi_producto_mayusculas ON practico1a.producto;
DROP FUNCTION IF EXISTS practico1a.fn_producto_mayusculas();

CREATE OR REPLACE FUNCTION practico1a.fn_producto_mayusculas()
RETURNS TRIGGER
LANGUAGE plpgsql
AS $$
BEGIN
    NEW.descripcion := UPPER(BTRIM(NEW.descripcion));
    RETURN NEW;
END;
$$;

CREATE TRIGGER bi_producto_mayusculas
BEFORE INSERT ON practico1a.producto
FOR EACH ROW
EXECUTE FUNCTION practico1a.fn_producto_mayusculas();
```

Por qué funciona

- Es un trigger BEFORE INSERT: la modificación de NEW.descripcion ocurre *antes* de que la fila se grabe físicamente, así el valor final es la descripción en mayúsculas y sin espacios al borde.
- En MySQL se asigna con SET NEW.col = ...; en PostgreSQL se asigna directamente con NEW.col := ... y se retorna NEW. Si la función olvida el RETURN NEW, el INSERT se cancela silenciosamente.
- TRIM/BTRIM adicional limpia espacios al inicio y al final, blindando contra entradas con formato sucio.

En MySQL 8+

Cubierto en la *Solución*: el inciso pide ambas bases (MySQL y PostgreSQL) y la sección incluye las dos.

6.2. b) Auditoría de cambios en producto.cantidad

Solución

La auditoría debe registrar cod_producto, el movimiento (NEW.cantidad - OLD.cantidad), la fecha y el usuario *de la aplicación cliente*. Como el motor no tiene forma de conocer ese usuario por sí mismo, la aplicación lo provee a través de una variable de sesión que el trigger lee.

MySQL 8+:

```
USE practico1a_mysql;

CREATE TABLE IF NOT EXISTS auditoriaProducto (
    id_auditoria          BIGINT AUTO_INCREMENT PRIMARY KEY,
    cod_producto          INT NOT NULL,
    movimiento           INT NOT NULL,
    fecha_actualizacion   DATETIME NOT NULL,
    usuario_aplicacion    VARCHAR(100) NOT NULL,
    CONSTRAINT fk_auditoria_producto
        FOREIGN KEY (cod_producto) REFERENCES producto(cod_producto)
);

DROP TRIGGER IF EXISTS au_producto_auditoria_cantidad;
DELIMITER $$
CREATE TRIGGER au_producto_auditoria_cantidad
AFTER UPDATE ON producto
FOR EACH ROW
BEGIN
    DECLARE v_usuario_app VARCHAR(100);

    IF NEW.cantidad <> OLD.cantidad THEN
        -- La aplicacion debe setear esta variable de sesion:
        -- SET @app_user_code = 'VENDEDOR_APP_01';
        SET v_usuario_app = @app_user_code;

        IF v_usuario_app IS NULL OR TRIM(v_usuario_app) = '' THEN
            SIGNAL SQLSTATE '45000'
            SET MESSAGE_TEXT =
                'Debe definir @app_user_code en la sesion para auditar';
        END IF;

        INSERT INTO auditoriaProducto (
            cod_producto, movimiento,
            fecha_actualizacion, usuario_aplicacion
        )
        VALUES (
```

```

        NEW.cod_producto,
        NEW.cantidad - OLD.cantidad,
        NOW(),
        v_usuario_app
    );
END IF;
END$$
DELIMITER ;

```

PostgreSQL 14+:

```

SET search_path TO practico1a;

CREATE TABLE IF NOT EXISTS practico1a.auditoria_producto (
    id_auditoria          BIGSERIAL PRIMARY KEY,
    cod_producto          INTEGER NOT NULL
        REFERENCES practico1a.producto(cod_producto),
    movimiento           INTEGER NOT NULL,
    fecha_actualizacion   TIMESTAMP NOT NULL,
    usuario_aplicacion    VARCHAR(100) NOT NULL
);

DROP TRIGGER IF EXISTS au_producto_auditoria_cantidad
ON practico1a.producto;
DROP FUNCTION IF EXISTS practico1a.fn_auditoria_producto_cantidad();

CREATE OR REPLACE FUNCTION practico1a.fn_auditoria_producto_cantidad()
RETURNS TRIGGER
LANGUAGE plpgsql
AS $$
DECLARE
    v_usuario_app TEXT;
BEGIN
    IF NEW.cantidad <> OLD.cantidad THEN
        -- La aplicacion debe setear esta variable de sesion:
        -- SET app.user_code = 'VENDEDOR_APP_01';
        v_usuario_app := current_setting('app.user_code', true);

        IF v_usuario_app IS NULL OR BTRIM(v_usuario_app) = '' THEN
            RAISE EXCEPTION
                'Debe definir app.user_code en la sesion para auditar';
        END IF;

        INSERT INTO practico1a.auditoria_producto (
            cod_producto, movimiento,
            fecha_actualizacion, usuario_aplicacion
        )
        VALUES (
            NEW.cod_producto,
            NEW.cantidad - OLD.cantidad,
            NOW(),
            v_usuario_app
        );
    END IF;

    RETURN NEW;
END;
$$;

CREATE TRIGGER au_producto_auditoria_cantidad
AFTER UPDATE OF cantidad ON practico1a.producto
FOR EACH ROW
EXECUTE FUNCTION practico1a.fn_auditoria_producto_cantidad();

```

Por qué funciona

- **Por qué AFTER UPDATE y no BEFORE.** La auditoría solo tiene sentido cuando la modificación ya ocurrió. Si un CHECK o un trigger anterior aborta el UPDATE, no debe quedar registro: con AFTER UPDATE, ambos motores garantizan que el trigger solo dispara cuando la operación es exitosa.
- El IF NEW.cantidad <>OLD.cantidad ahorra trabajo cuando el UPDATE no toca esa columna (por ejemplo, si el cambio es solo en precio). En PostgreSQL, la cláusula AFTER UPDATE OF cantidad es aún más eficiente: el trigger ni siquiera se invoca si esa columna no cambia.
- **Usuario de aplicación.** El motor no conoce el usuario lógico de la app; la aplicación lo inyecta en una variable de sesión (@app_user_code en MySQL, current_setting('app.user_code') en PostgreSQL) que el trigger lee. Si la app olvida setearla, el trigger aborta con SIGNAL/RAISE EXCEPTION, forzando a configurarla.

En MySQL 8+

Cubierto en la *Solución* (el inciso pide ambas bases).

Prueba

```
-- MySQL
SET @app_user_code = 'VENDEDOR_APP_01';
UPDATE producto SET cantidad = cantidad + 5 WHERE cod_producto = 101;
SELECT * FROM auditoriaProducto ORDER BY id_auditoria DESC;

-- PostgreSQL
SET app.user_code = 'VENDEDOR_APP_01';
UPDATE practico1a.producto SET cantidad = cantidad + 5 WHERE cod_producto = 101;
SELECT * FROM practico1a.auditoria_producto ORDER BY id_auditoria DESC;
```

6.3. c) Autonumerado de parquimetro

Solución

MySQL 8+ (*practico1b_mysql*): basta declarar la columna como AUTO_INCREMENT.

```
USE practico1b_mysql;

ALTER TABLE parquimetro
    MODIFY id_parquimetro INT NOT NULL AUTO_INCREMENT;

-- Despues, omitir la columna en el INSERT:
INSERT INTO parquimetro (calle, altura) VALUES ('Colon', 1500);
SELECT * FROM parquimetro ORDER BY id_parquimetro;
```

Oracle XE 10/11 (SEQUENCE + TRIGGER): no hay IDENTITY hasta Oracle 12c; el patrón es secuencia + trigger.

```
-- Limpieza opcional para re-ejecutar el script.
BEGIN EXECUTE IMMEDIATE 'DROP TRIGGER bi_parquimetro_autonumerado';
EXCEPTION WHEN OTHERS THEN NULL; END;
/
BEGIN EXECUTE IMMEDIATE 'DROP SEQUENCE seq_parquimetro';
```

```

EXCEPTION WHEN OTHERS THEN NULL; END;
/

-- Crear la secuencia arrancando despues del MAX existente,
-- para no chocar con datos cargados.
DECLARE
    v_start NUMBER;
BEGIN
    SELECT NVL(MAX(id_parquimetro), 0) + 1
        INTO v_start
        FROM parquemetro;

    EXECUTE IMMEDIATE
        'CREATE SEQUENCE seq_parquimetro START WITH ' || v_start ||
        ' INCREMENT BY 1 NOCACHE NOCYCLE';
END;
/

CREATE OR REPLACE TRIGGER bi_parquimetro_autonumerado
BEFORE INSERT ON parquemetro
FOR EACH ROW
WHEN (NEW.id_parquimetro IS NULL)
BEGIN
    SELECT seq_parquimetro.NEXTVAL
        INTO :NEW.id_parquimetro
        FROM dual;
END;
/

```

Por qué funciona

- En MySQL, `AUTO_INCREMENT` es una propiedad de la columna que el motor administra automáticamente; el cliente puede omitir la columna en el `INSERT` y MySQL la rellena.
- En Oracle 10/11 XE el mismo efecto requiere dos objetos cooperando: la `SEQUENCE` produce valores únicos crecientes, y el `TRIGGER BEFORE INSERT` los inyecta en `:NEW.id_parquimetro` cuando el cliente no proporciona uno explícito.
- La cláusula `WHEN (NEW.id_parquimetro IS NULL)` evita pisar los `INSERT` con ID explícito, típicos en cargas de datos de prueba.
- El bloque PL/SQL que arranca la secuencia con `MAX + 1` sirve para que los IDs no choquen con los datos ya cargados en el Práctico 1.b.

En MySQL 8+

Cubierto en la *Solución*: el inciso pide explícitamente MySQL para la base de estacionamiento (y, además, plantea la pregunta abierta de cómo resolverlo en Oracle).

Conclusión

Los cinco ejercicios cubren el espectro de programación del lado servidor: *triggers* para reglas que dependen de varias tablas (Ej. 1), *funciones* con cálculo manual y parámetros IN/OUT (Ej. 2), *procedimientos* con validación, transaccionalidad y mensajes por pantalla (Ej. 3), *procedimientos* con seguridad por invocador (Ej. 4) y *triggers* de normalización, auditoría y autonumerado (Ej. 5).

Puntos importantes para el parcial:

- **BEFORE/AFTER + INSERT/UPDATE/DELETE** es la matriz básica del trigger; NEW y OLD dan acceso a la fila nueva y vieja.
- Para *abortar* desde el trigger: **SIGNAL SQLSTATE '45000'** en MySQL, **RAISE EXCEPTION** en PostgreSQL, **RAISE_APPLICATION_ERROR** en Oracle.
- Las *funciones* se asocian a **RETURN** de un valor; los *procedimientos* se invocan con **CALL** (MySQL, PostgreSQL) o en bloques **BEGIN ... END** (Oracle) y pueden tener IN, OUT e INOUT. En MySQL, los parámetros OUT solo existen en procedimientos.
- Para *auditar el usuario de aplicación*: variables de sesión (**@var** en MySQL, **current_setting(...)** en PostgreSQL).
- **SQL SECURITY INVOKER/DEFINER** en MySQL es la pieza que conecta el Práctico 3 con el Práctico 2 (DCL).
- **AUTO_INCREMENT** (MySQL) e **IDENTITY** (PostgreSQL/Oracle 12+) ahorran código; en Oracle 10/11 XE el patrón es **SEQUENCE + TRIGGER BEFORE INSERT**.